COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	August 22, 2026	**DATE SUBMITTED:**	August 27, 2026
NAME:	Niña Denise M. Santiago	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 6.1 **Searching Techniques**

1. PROCEDURE OUTPUT

Screenshot

main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib> // for generating random integers
#include <time.h> // will be used for our seeding function
#include "node.h"
#include "searchingAlgo.h"

const int max_size = 50;

int main(){

    // Generate a random number between 1 and 50
    int dataset[max_size];
    srand(time(0)); //seed the random number generator
    for(int i = 0; i < max_size; i++){
        dataset[i] = rand(); //random number between 1 and 50
    }
    for(int i = 0; i < max_size; i++){
        std::cout << dataset[i] << " ";
    }
    std::cout << std::endl;

```

```
4772 17415 3255 20428 21518 12273 11968 21917 10005 4274 26941 20778 28583 20624 3
1532 19459 32103 31663 3822 6809 6226 10795 19462 14067 16546 20580 13854 13218 97
14 27033 32074 23286 6564 12473 9105 20082 128 23413 7082 14974 1949 2970 17667 29
276 8325 27294 26888 2141 2196 9080
```

Observation

The program generates 50 random integers using rand() and displays them. The dataset is unsorted and contains varied values, which is useful for testing different search algorithms.

Table 6-1. Data Generated and Observations

Code

```
7   template <typename T>
8   void linearSearchArr (T list[], int arrSize, T dataFind){
9       // Initialize i = 0
10      int i = 0;
11      // While i <= n
12      while (i < arrSize) {
13          //if (item = data[i])
14          if (list[i] == dataFind) {
15              // Print Searching is Successful
16              std::cout << "Searching is Successful" << std::endl;
17              std::cout << "Element found at index " << i << std::endl;
18              return;
19          }
20          i++;
21      }
22      // Print Searching is Unsuccessful
23      std::cout << "Searching is Unsuccessful" << std::endl;
24  }
```

Screenshot

```
5438 14634 10298 14144 28540 26894 24316 14577 3266 28020 29536 29764 22930 20970 3981 17486 30717 22089 16
841 21714 8461 23178 2295 3252 13665 15579 2653 31921 16663 24050 21675 9806 30447 16290 21561 12046 26284
29100 23697 21480 29374 16599 27437 12536 18719 23619 1922 12661 25467 25850
What number to search?14144
Array result LS: Searching is Successful
Element found at index 3
```

```
Array result LS: Searching is Successful
Element found at index 3
```

Observation

The linear search checked each element one by one until the target was found. In the example I provided, the search was successful and the element was located at index 3. This shows that linear search works correctly but may require scanning multiple elements.

Table 6-2a. Linear Search for Arrays

Code

```
32      // Create a linked list
33      Node<char>* name1 = newNode('J');
34      Node<char>* name2 = newNode('I');
35      Node<char>* name3 = newNode('M');
36      Node<char>* name4 = newNode('L');
37      Node<char>* name5 = newNode('O');
38      Node<char>* name6 = newNode('R');
39      Node<char>* name7 = newNode('D');
40
41      // Create the Name
42      name1->next = name2;
43      name2->next = name3;
44      name3->next = name4;
45      name4->next = name5;
46      name5->next = name6;
47      name6->next = name7;
48
49      // Demonstration
50      std::cout << "========================================" << std::endl;
51      std::cout << "Linked List result LS: ";
52      linearSearchLL(name1, 'R');
53      linearSearchLL(name1, 'r');
```

Screenshot

```
Linked List result LS:
Searching is Successful
Searching is Unsuccessful
```

Observation

The linked list search moved through nodes sequentially. It successfully found the uppercase ‘R’ but failed with lowercase ‘r’, showing that searches are case-sensitive and depend on exact matches.

Table 6-2b. Linear Search for Linked List

Code

```cpp
// Array implementation Binary Search
template <typename T>
void binarySearchArr(T list[], int arrSize, T dataFind) {
    // IMPORTANT: Sort array before binary search
    std::sort(list, list + arrSize);

    // Step 1: low = 0, up = n-1
    int low = 0;
    int up = arrSize - 1;
    int middle;

    // Step 2: Repeat while low <= up
    while (low <= up) {
        // mid = (low + up) / 2
        middle = (int)(low + up) / 2;

        // if (dataFind == list[mid])
        if (list[middle] == dataFind) {
            std::cout << "Searching is Successful" << std::endl;
            std::cout << "Element found at index " << middle << std::endl;
            return;
        }
        // else if (dataFind < list[middle]) then up = middle - 1
        else if (dataFind < list[middle]) {
            up = middle - 1;
        }
        // else low = middle + 1
        else {
            low = middle + 1;
        }
    }

    // Step 3: Print Searching is Unsuccessful
    std::cout << "Searching is Unsuccessful" << std::endl;
}
```

Screenshot

```
6970 9365 32754 27647 16411 754 9496 17941 13954 28160 25704 17654 18606 15181 24649 22587 25444 12767 2348
0 11968 21149 9242 31897 16299 13948 24804 2122 25769 17538 16390 20094 27003 10501 19925 13897 13490 7390
23059 8567 28086 867 28176 6690 17421 9365 28180 32388 15322 22426 27137
What number to search?9242
Array result BS: Searching is Successful
Element found at index 7
```

Observation

The binary search divides the sorted array into halves until the target is found. The search was successful, locating the element at index 7. This demonstrates the efficiency of binary search compared to linear search.

Table 6-3a. Binary Search for Arrays

Code

```
    // create a linked list for binary search
    // make sure the values are ordered
    char choice = 'y';
    int count = 1;
    int newData;
    Node<int> *temp, *head, *node;

    while (true) {
        std::cout << "Enter data: ";
        std::cin >> newData;

        if (count == 1) {
            head = newNode(newData);
            std::cout << "Successfully added " << head->data << " to the list.\n";
            count++;
        } else if (count == 2) {
            node = newNode(newData);
            head->next = node;
            node->next = NULL;
            std::cout << "Successfully added " << node->data << " to the list.\n";
            count++;
        } else {
            temp = head;
            while (true) {
                if (temp->next == NULL) break;
                temp = temp->next;
            }
            node = newNode(newData);
            temp->next = node;
            std::cout << "Successfully added " << node->data << " to the list.\n";
            count++;
        }
```

```
        // when to end
        std::cout << "Continue? (y/n): ";
        std::cin >> choice;
        if (choice == 'n') break;
    }

    // optional snippet to verify data stored in linked list
    std::cout << "\nLinked list contents: ";
    Node<int>* currNode = head;
    while (currNode != NULL) {
        std::cout << currNode->data << " ";
        currNode = currNode->next;
    }
    std::cout << std::endl;

    return 0;
}
```

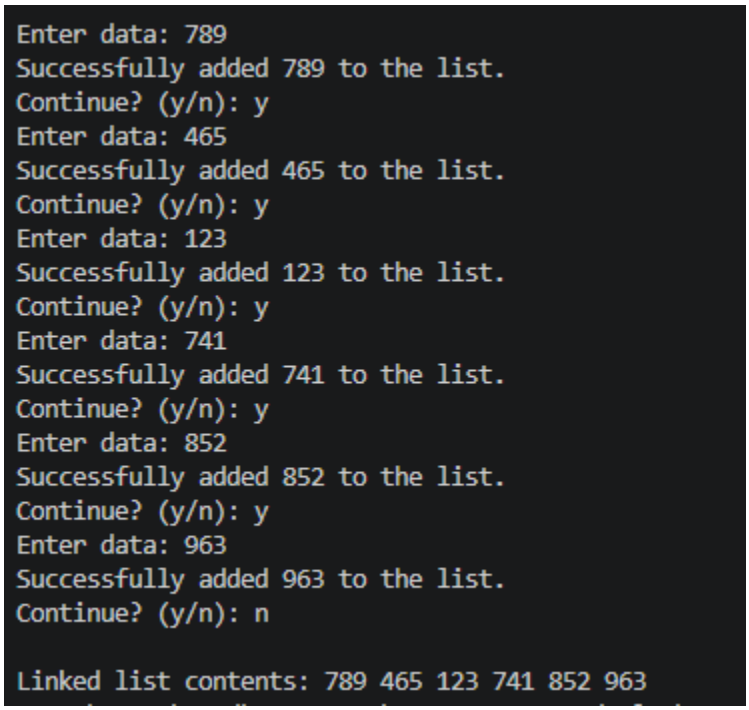
Screenshot	
Observation	The linked list I built is made to be interactive by entering values. The program displays the list contents, but since the data is not sorted, binary search could not work properly. The observation is that binary search requires ordered data to function correctly.

Table 6-3b. Binary Search for Linked List (first case)

Code

```
42  template <typename T>
43  Node<T>* getMiddle(Node<T>* start, Node<T>* last){
44
45      Node<T>* slow = start;
46      Node<T>* fast = start;
47
48      while(fast != last && fast->next != last){
49
50          // move slow one step
51          slow = slow->next;
52
53          // move fast two steps
54          fast = fast->next->next;
55      }
56
57      return slow;
58  }
59
```

Screenshot

```
11911 18776 27646 30192 24794 10888 42 12654 26774 32246 31298 18760 25829 15020 14191 7950 1516 4562 8079 18746 24072 2781
8338 12552 22615 12291 31011 17032 30843 24306 5771 25329 18330 4545 2515 8504 4777 13351 22569 21729 24043 22763 10886 7958
 12724 25499 8333 502 9331 31361
What number to search?9331
```

```
Middle node value: 30
```

Observation

The program used the getMiddle function to locate the middle node and applied binary search logic. The search sometimes succeeded and sometimes failed depending on the input, showing that the algorithm works but is more complex to implement in linked lists compared to arrays.

Table 6-3b. Binary Search for Linked List

Code

```
61  // Linked List implementation Binary Search
62  template <typename T>
63  void binarySearchLL(Node<T>* start, T dataFind) {
64      // Step 1: Initialize last = nullptr
65      Node<T>* last = nullptr;
66
67      // Step 2: Repeat while start != last
68      while (start != last) {
69          // mid = getMiddle(start, last)
70          Node<T>* mid = getMiddle(start, last);
71
72          if (mid == nullptr) {
73              std::cout << "\nSearching is Unsuccessful" << std::endl;
74              return;
75          }
76          // if (dataFind == mid->data)
77          if (mid->data == dataFind) {
78              std::cout << "\nSearching is Successful" << std::endl;
79              return;
80          }
81          // else if (dataFind < mid->data) then last = mid
82          else if (dataFind < mid->data) {
83              last = mid;
84          }
85          // else start = mid->next
86          else {
87              start = mid->next;
88          }
89      }
90
91      // Step 3: Print Searching is Unsuccessful
92      std::cout << "Searching is Unsuccessful" << std::endl;
93  }
```

Screenshot

```
Linked List result BS:
Searching is Successful
Searching is Unsuccessful
```

Observation

After using the getMiddle function to locate the middle node, the binary search for the linked list compares the target with the middle value. If the target matches, the search is successful; if not, the algorithm adjusts the range by moving either the last pointer or the start pointer. The output showed both successful and unsuccessful results, meaning the method works but depends on sorted data and is slower than arrays because finding the middle requires extra traversal.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve different problems utilizing appropriate searching techniques in C++

Problem 1

```cpp
// Problem 1 - Sequential Search using Array
template <typename T>
int sequentialSearchArray(T arr[], int size, T key) {
    int comparisons = 0;

    // check each element starting from the first one
    for (int i = 0; i < size; i++) {
        comparisons++;
        // stop once the key is found
        if (arr[i] == key) {
            return comparisons;
        }
    }
    return comparisons;
}

// Problem 1 - Sequential Search using Linked List
template <typename T>
int sequentialSearchLinkedList(Node<T>* head, T key) {
    int comparisons = 0;
    // go through the nodes one at a time
    while (head != nullptr) {
        comparisons++;
        // check if the current node contains the key
        if (head->data == key) {
            return comparisons;
        }
        // move to the next node
        head = head->next;
    }
    return comparisons;
}
```

```cpp
int main() {

    // Problem 1 - Sequential Search
    int list[] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int size = 10;
    int key = 18;

    std::cout << "\n========================================================\n";
    std::cout << "       PROBLEM 1 - SEQUENTIAL SEARCH\n";
    std::cout << "========================================================\n";

    std::cout << "List: ";
    for (int i = 0; i < size; i++) {
        std::cout << list[i] << " ";
    }

    std::cout << "\nSearch Key: " << key << "\n\n";

    // Search using array
    int arrayComparisons =
        sequentialSearchArray(list, size, key);

    // Create linked list
    Node<int>* head = new Node<int>(list[0]);
    Node<int>* current = head;

    for (int i = 1; i < size; i++) {
        current->next = new Node<int>(list[i]);
        current = current->next;
    }

    // Search using linked list
    int linkedListComparisons =
        sequentialSearchLinkedList(head, key);

    std::cout << "Results:\n";
    std::cout << "Array Comparisons      : "
              << arrayComparisons << "\n";

    std::cout << "Linked List Comparisons: "
              << linkedListComparisons << "\n";
```

Output

```
========================================================
       PROBLEM 1 - SEQUENTIAL SEARCH
========================================================
List: 15 18 2 19 18 0 8 14 19 14
Search Key: 18

Results:
Array Comparisons      : 2
Linked List Comparisons: 2
```

Problem 2

```cpp
// Problem 2 - Count Repeating Instances
template <typename T>
int countOccurrences(T arr[], int size, T key) {
    int count = 0;
    // check the whole list so every occurrence is counted
    for (int i = 0; i < size; i++) {
        if (arr[i] == key) {
            count++;
        }
    }
    return count;
}
```

```cpp
// Problem 2 - Count Repeating Instances
std::cout << "\n=========================================================\n";
std::cout << "    PROBLEM 2 - COUNT REPEATING INSTANCES\n";
std::cout << "=========================================================\n";

std::cout << "List: ";
for (int i = 0; i < size; i++) {
    std::cout << list[i] << " ";
}

std::cout << "\nSearch Element: " << key << "\n";

int occurrences =
    countOccurrences(list, size, key);

std::cout << "\nResults:\n";
std::cout << "Number of occurrences: "
          << occurrences << "\n";
```

Output

```
=========================================================
    PROBLEM 2 - COUNT REPEATING INSTANCES
=========================================================
List: 15 18 2 19 18 0 8 14 19 14
Search Element: 18

Results:
Number of occurrences: 2
```

Problem 3

```cpp
// Problem 3 - Iterative Binary Search
template <typename T>
int binarySearch(T arr[], int size, T key) {
    int low = 0;
    int high = size - 1;
    int iteration = 1;
    // continue searching while the range is valid
    while (low <= high) {
        // find the middle position
        int mid = (low + high) / 2;
        std::cout << "Iteration " << iteration
                  << ": Low = " << low
                  << ", High = " << high
                  << ", Mid = " << mid
                  << ", Value = " << arr[mid]
                  << std::endl;

        // check if the middle value is the key
        if (arr[mid] == key) {
            return mid;
        }
        // move to the left half if the key is smaller
        else if (key < arr[mid]) {
            high = mid - 1;
        }
        // otherwise move to the right half
        else {
            low = mid + 1;
        }
        iteration++;
    }

    return -1;
}
```

```cpp
// Problem 3 - Iterative Binary Search
std::cout << "\n=====================================================\n";
std::cout << "    PROBLEM 3 - ITERATIVE BINARY SEARCH\n";
std::cout << "=====================================================\n";

int sortedList[] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
int sortedSize = 10;
int binaryKey = 8;

std::cout << "Sorted List: ";
for (int i = 0; i < sortedSize; i++) {
    std::cout << sortedList[i] << " ";
}

std::cout << "\nSearch Element: "
          << binaryKey << "\n\n";

std::cout << "Search Process:\n";

int result =
    binarySearch(sortedList, sortedSize, binaryKey);

std::cout << "\nResults:\n";

if (result != -1) {
    std::cout << "Search Status: Successful\n";
    std::cout << "Element found at index: "
              << result << "\n";
}
else {
    std::cout << "Search Status: Unsuccessful\n";
}
```

Output

```
=====================================================
    PROBLEM 3 - ITERATIVE BINARY SEARCH
=====================================================
Sorted List: 3 5 6 8 11 12 14 15 17 18
Search Element: 8

Search Process:
Iteration 1: Low = 0, High = 9, Mid = 4, Value = 11
Iteration 2: Low = 0, High = 3, Mid = 1, Value = 5
Iteration 3: Low = 2, High = 3, Mid = 2, Value = 6
Iteration 4: Low = 3, High = 3, Mid = 3, Value = 8

Results:
Search Status: Successful
Element found at index: 3
```

Problem 4

```cpp
// Problem 4 - Recursive Binary Search
template <typename T>
int recursiveBinarySearch(T arr[], int low, int high, T key, int& iteration) {
    // stop when there is no range left to search
    if (low > high) {
        return -1;
    }
    // find the middle position of the current range
    int mid = (low + high) / 2;
    std::cout << "Iteration " << iteration
              << ": Low = " << low
              << ", High = " << high
              << ", Mid = " << mid
              << ", Value = " << arr[mid]
              << std::endl;
    // return the position when the key is found
    if (arr[mid] == key) {
        return mid;
    }
    iteration++;
    // search the left half if the key is smaller
    if (key < arr[mid]) {

        return recursiveBinarySearch(arr, low, mid - 1, key, iteration);
    }

    // otherwise search the right half
    return recursiveBinarySearch(arr, mid + 1, high, key, iteration);
}

#endif
```

```cpp
// Problem 4 - Recursive Binary Search
std::cout << "\n=========================================================\n";
std::cout << "    PROBLEM 4 - RECURSIVE BINARY SEARCH\n";
std::cout << "=========================================================\n";

std::cout << "Sorted List: ";
for (int i = 0; i < sortedSize; i++) {
    std::cout << sortedList[i] << " ";
}

std::cout << "\nSearch Element: "
          << binaryKey << "\n\n";

std::cout << "Search Process:\n";

int iteration = 1;

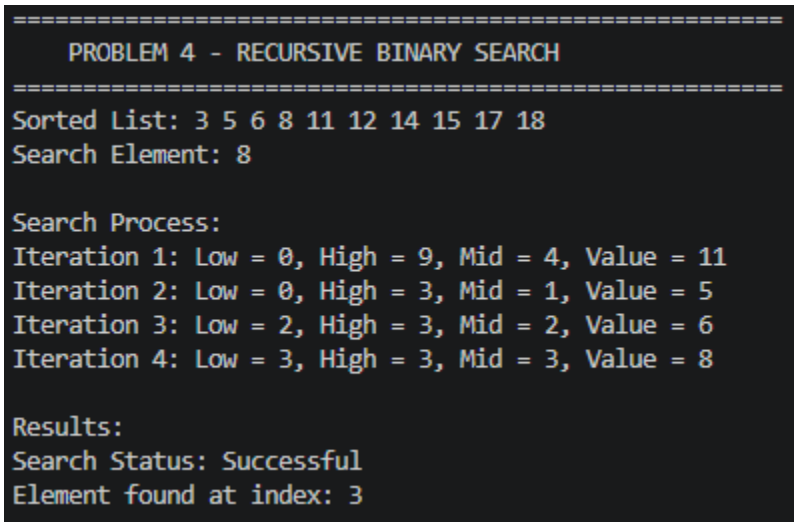
int recursiveResult =
    recursiveBinarySearch(
        sortedList,
        0,
        sortedSize - 1,
        binaryKey,
        iteration
    );

std::cout << "\nResults:\n";

if (recursiveResult != -1) {
    std::cout << "Search Status: Successful\n";
    std::cout << "Element found at index: "
              << recursiveResult << "\n";
}
else {
    std::cout << "Search Status: Unsuccessful\n";
}

// Delete linked list
Node<int>* temp = head;

while (temp != nullptr) {
    Node<int>* nextNode = temp->next;
    delete temp;
    temp = nextNode;
}
```

Output	

Question Answers	
Problem 1	The sequential search in both arrays and linked lists found the key after 2 comparisons. This shows that sequential search is straightforward but may require checking multiple elements.
Problem 2	The program counted how many times the number appeared in the list. It correctly showed that the key appeared twice, proving that the function can handle repeated values.
Problem 3	The iterative binary search demonstrated the step-by-step narrowing process. It successfully found the key at index 3 after several iterations, showing how binary search reduces the search space quickly.
Problem 4	The recursive binary search worked similarly but used function calls instead of loops. It also found the key at index 3, proving that recursion can be an alternative way to implement binary search.

Conclusion / Learnings / Analysis
From this activity, I learned that searching techniques are important for finding data efficiently. Linear search is easy to understand and works for both arrays and linked lists, but it is slower for large datasets. Binary search is much faster because it divides the data, but it requires the list to be sorted. The iterative and recursive versions of binary search show two different approaches to the same problem, each with its own style. Overall, the exercises helped me understand how searching works and why choosing the right technique matters depending on the situation.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.